ENTERPRISE DATA LANDSCAPE & ARCHITECTURE BLUEPRINT

SAP · JD Edwards · FactoryTalk → Azure Data Lake → Databricks Lakehouse

ETL Pipelines · Unity Catalog · CDC/CDF · Data Governance · ML/AI · Agentic Readiness

Ball Corporation — Technical Architecture & Design Document

Confidential · May 2026

TABLE OF CONTENTS

1. SAP SOURCE LAYER & DATA EXTRACTION ARCHITECTURE

Ball Corporation's SAP landscape spans 10+ functional modules across ECC and S/4HANA instances. The extraction strategy must accommodate both batch (full/delta) and near-real-time CDC patterns for each module, using the appropriate SAP connector and extraction framework.

1.1 SAP Module Inventory

Module	Full Name	Key Tables	Extraction Method
FI	Financial Accounting	BKPF, BSEG, ACDOCA, SKA1/SKAT	SAP CDC via ADF
CO	Controlling	CSKS/CSKT, CEPC/CEPCT, COBK	ODP Delta Queues
SD	Sales & Distribution	VBAK, VBAP, KNA1, VBRK/VBRP	CDS Views + ODP
MM	Materials Management	EKKO, EKPO, MARA, MAKT, LFA1	Timestamp Delta
PP	Production Planning	AUFK, AFKO, AFPO, MAST, STKO	Full + Incremental
IBP	Integrated Planning	Forecast Views, Planning Areas	OData API Extract
QM	Quality Management	QALS, QAPO, QAVE	ADF SAP Table
PS	Project Systems	PROJ, PRPS	ADF SAP Table
PM	Plant Maintenance	AUFK (PM type), AFIH	ADF SAP Table
SCM	Supply Chain Mgmt	Logistics execution tables	SLT Replication

Figure 1.1 — SAP Module Extraction Matrix

1.2 Extraction Architecture Flow

The SAP CDC extraction pattern uses ADF's native SAP CDC source type rather than the legacy SAP Table connector. The CDC connector tracks deletes and updates automatically at the SAP application level, eliminating complex delta-tracking logic in Azure.

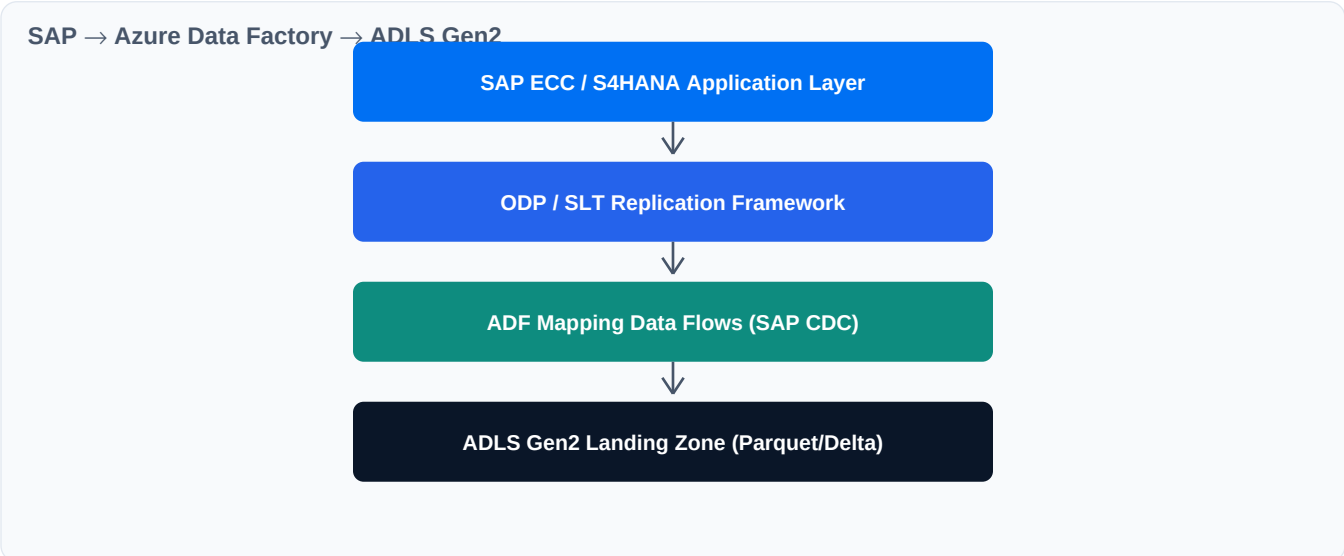

Figure 1.2 — SAP to Azure Data Lake Extraction Flow

1.3 Key Technical Best Practices

- **SAP CDC Pattern:** Always use the SAP CDC source type in ADF instead of the legacy SAP Table connector. CDC tracks deletes/updates automatically at the SAP application level.
- **SHIR Sizing:** Place the Self-Hosted Integration Runtime on an Azure VM geographically close to your SAP application server to minimize latency when pulling tables like BSEG or ACDOCA.
- **Service Principal Auth:** Set up Databricks External Locations using an Azure Service Principal or Managed Identity with Storage Blob Data Contributor access on ADLS Gen2.

2. AZURE DATA FACTORY + SHIR INGESTION DESIGN

Azure Data Factory serves as the primary orchestration and ingestion engine. The Self-Hosted Integration Runtime (SHIR) bridges the on-premises SAP landscape with Azure cloud services through a secure private link connection.

2.1 SHIR Architecture

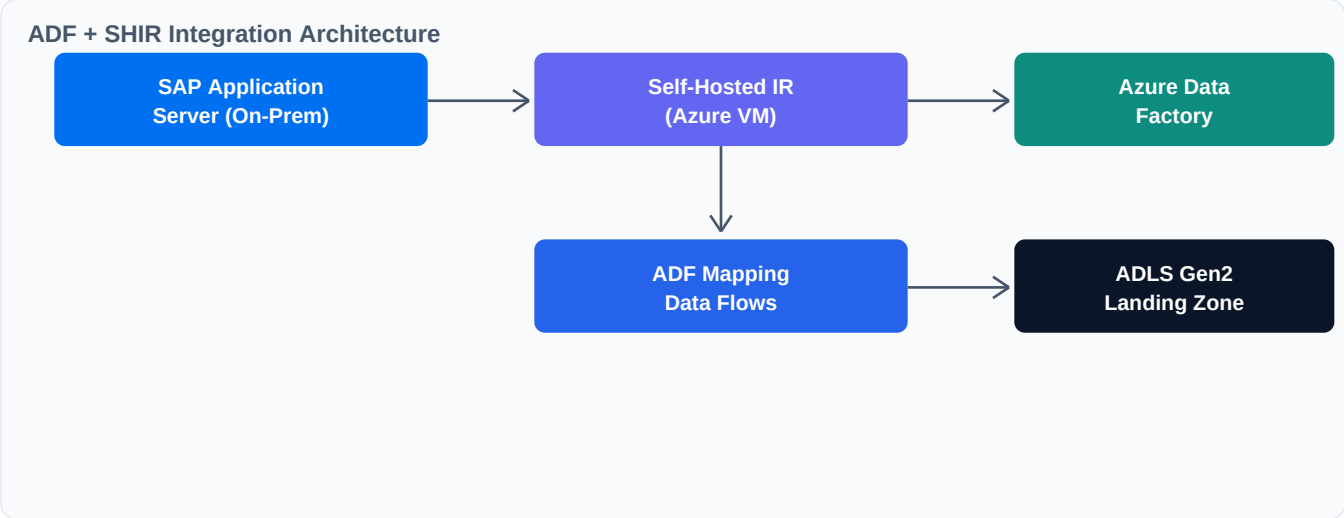

Figure 2.1 — Self-Hosted Integration Runtime Architecture

2.2 Pipeline Design Patterns

Pattern	Use Case	Trigger	Frequency
Full Load	Initial migration, small lookup tables	Schedule	One-time / Weekly
Delta (CDC)	Large transactional tables (ACDOCA, BSEG)	Schedule	Every 15-60 min
SLT Replication	Near-real-time change capture from SAP	Event-driven	Continuous
OData API	IBP forecasts, cloud-based SAP data	Schedule	Daily / Hourly

Figure 2.2 — ADF Pipeline Patterns

2.3 Step-by-Step Data Flow Sequence

- **Step 1 — Extract:** ADF initiates connection to SAP through SHIR using the ODP framework to request full or delta changes from SAP extractors (e.g., 0FI_ACDOCA_10 for Finance, 2LIS_11_VAHDR for Sales).
- **Step 2 — Stage:** ADF Mapping Data Flows transform raw SAP structures into optimized Parquet files and write them into the ADLS Gen2 landing zone container.
- **Step 3 — Ingest:** Databricks Auto Loader (cloudFiles) monitors the ADLS Gen2 container and incrementally ingests new files into Unity Catalog Bronze schema as managed Delta tables.
- **Step 4 — Refine:** Databricks Workflows or Delta Live Tables pick up Bronze tables, apply column typing, decode language-dependent strings, and join headers with line items into Silver schema.
- **Step 5 — Aggregate:** Cross-functional business rules combine modules into business-ready Gold tables for Power BI reporting and analytics.

3. AZURE DATA LAKE STORAGE GEN2 LANDING ZONE

ADLS Gen2 acts as the centralized cloud landing zone where all raw data from SAP, JD Edwards, and FactoryTalk is staged before ingestion into the Databricks Lakehouse. The storage follows a hierarchical namespace structure for efficient access control and performance.

3.1 Container Structure

Container	Purpose	Format	Access
raw-sap-landing	SAP CDC extracts from ADF	Parquet / Delta	ADF Service Principal
raw-jde-landing	JD Edwards JDBC batch pulls	Parquet	ADF Service Principal
raw-iot-landing	FactoryTalk sensor streams	JSON / Avro	IoT Hub / Event Hub
raw-api-landing	REST API + file-based sources	CSV / JSON	ADF / Functions
databricks-managed	Databricks managed tables	Delta Lake	Databricks MI
checkpoints	Auto Loader checkpoint state	System	Databricks MI

Figure 3.1 — ADLS Gen2 Container Layout

3.2 Security & Networking

- **Network Isolation:** Private endpoints for all storage accounts — no public internet access
- **Authentication:** Azure Managed Identity federation between ADF, Databricks, and ADLS Gen2
- **RBAC:** Storage Blob Data Contributor for Databricks service principal; Storage Blob Data Reader for analytics consumers
- **Access Control:** Hierarchical namespace enabled for fine-grained directory-level ACLs

4. DATABRICKS LAKEHOUSE — MEDALLION ARCHITECTURE

The Databricks Lakehouse implements a three-tier Medallion Architecture (Bronze → Silver → Gold) on top of Delta Lake. This pattern provides progressive data refinement from raw ingestion to business-ready analytics.

4.1 Medallion Layers

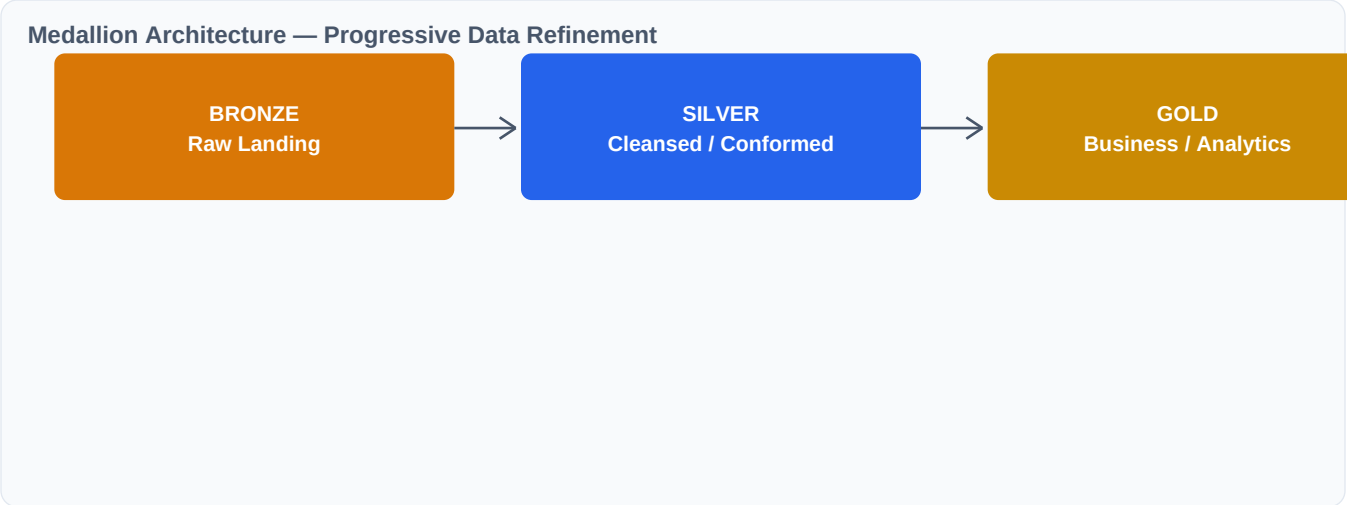

Figure 4.1 — Medallion Lakehouse Architecture

Layer	Purpose	Data Characteristics	Key Operations
Bronze	Raw data landing	As-is from source, schema-on-read, full + incremental loads	Auto Loader ingestion, no transformations
Silver	Cleansed & conformed	Deduplicated, typed, business keys aligned, SCD Type 2 history	Column typing, SAP text joins, DQ expectations
Gold	Business-ready	Star schema dimensional models, aggregated KPIs, feature tables	Cross-module joins, business rule aggregations

Figure 4.2 — Medallion Layer Specification

4.2 Transformation Logic by SAP Module

- **Finance:** Join ACDOCA (S/4) or BKPF/BSEG (ECC) with text tables SKA1/SKAT filtered by SPRAS='E' to resolve G/L account descriptions.
- **Sales:** Flatten VBAK (header) + VBAP (item) into unified sd_orders. Convert NETWR → net_value, WAERK → currency, VKORG → sales_organization.
- **Procurement:** Combine EKKO + EKPO into mm_purchase_orders. Resolve vendor numbers from LFA1 into company names in mm_vendors.
- **Manufacturing:** Use PySpark GraphFrames to create exploded, flattened representations of multi-level BOMs from MAST/STKO/STPO.
- **Planning:** Store IBP time-series forecasts as snapshots to track forecast accuracy vs actuals over time.

5. UNITY CATALOG — MULTI-CATALOG DESIGN

For Ball Corporation's enterprise scale, the recommended approach is the Multi-Catalog Pattern which establishes strict operational boundaries between ingestion infrastructure, domain data products, and cross-domain analytics.

5.1 Three-Tier Multi-Catalog Architecture

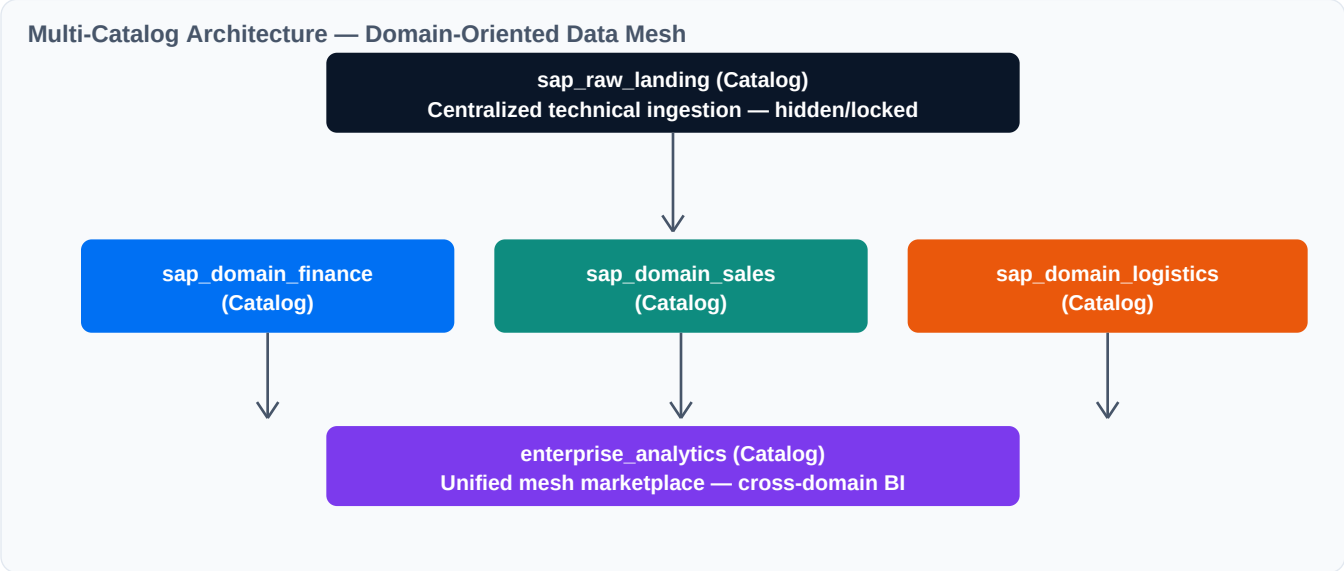

Figure 5.1 — Unity Catalog Multi-Catalog Pattern

5.2 Option A: Multi-Schema Prefix (Single Catalog)

In this layout, business domains are kept exactly as designed but use standard prefixes (b_, s_, g_) within each schema to separate Raw, Cleaned, and Reporting data:

Schema	Prefix	Table Example	Description
finance	b_	b_acdoca	Raw Universal Journal from SAP
finance	s_	s_fi_gl_postings	Cleaned, typed journal entries
finance	g_	g_financial_statement	Gold reporting matrix
sales	b_	b_vbak / b_vbap	Raw header/item tables
sales	s_	s_sd_orders	Unified sales orders
sales	g_	g_order_to_cash_kpi	Aggregated business KPIs

Figure 5.2 — Multi-Schema Prefix Pattern

5.3 Option B: Multi-Catalog (Recommended for Large Enterprise)

Completely clean schemas where data users only see final tables. Split into three separate catalogs:

Catalog	Purpose	Access Level	Example Tables

sap_enterprise_raw	ADF raw landing	Restricted — service accounts only	bkpf, bseg, acdoca, vbak, vbap, kna1
sap_enterprise_refined	Cleaned & joined via Databricks Workflows	Domain data engineers & scientists	fi_accounts, fi_gl, sd_orders, sd_customers
sap_enterprise_analytics	Business KPIs for Power BI / Tableau	All authorized business users	cash_flow_forecast, sales_funnel_velocity

Figure 5.3 — Multi-Catalog Pattern for Large Enterprise

5.4 Unity Catalog Security Implementation

Unity Catalog allows security at Catalog, Schema, Table, Column, and Row levels:

```
REVOKE ALL PRIVILEGES ON CATALOG sap_enterprise FROM ACCOUNT USERS;
GRANT USAGE ON CATALOG sap_enterprise TO finance_analysts_group;
GRANT USAGE, SELECT ON SCHEMA sap_enterprise.finance TO finance_analysts_group;
```

Column-level masking for PII protection:

```
CREATE FUNCTION sap_enterprise.sales.customer_mask(email STRING)
RETURN CASE WHEN is_account_group_member('sales_managers') THEN email ELSE 'REDACTED' END;
```

6. DOMAIN-ORIENTED DATA MESH OPERATING MODEL

The Data Mesh model decentralizes data ownership into specific business domains. Each domain team owns their catalog, pipelines, and data definitions. Data is treated as a product with published contracts and SLAs.

6.1 Domain Catalog Blueprint

Domain Catalog	Owner	Refined Schema	Products Schema (Public)
sap_domain_finance	Corporate Finance Data Team	ACDOCA, BSEG, CEPC joins	fi_accounts, fi_gl_transactions, co_cost_centers, co_profit_centers
sap_domain_sales	Commercial Ops Data Team	VBAK, VBAP, VBRK flattening	sd_orders, sd_customers, sd_billing
sap_domain_logistics	Supply Chain & Procurement	EKKO, MARA, AUFK, QALS maps	mm_materials, mm_purchase_orders, pp_production_orders, qm_inspections
sap_domain_projects	Enterprise PMO Data Team	PROJ, PRPS WBS mapping	ps_projects

Figure 6.1 — Domain Catalog Ownership Matrix

6.2 Data Mesh Principles

- **Federated Ownership:** Catalog owners are set to domain data steward groups (e.g., Finance Data Steward owns sap_domain_finance). Domain teams manage their own lifecycle.
- **Metadata Tagging:** Catalogs, schemas, and tables are decorated with tags (tier:production, classification:confidential, domain:finance) and markdown documentation.
- **Cross-Domain Composition:** When business units need cross-domain reports (e.g., Order-to-Cash requiring sales + finance), they build composite views in an enterprise_analytics consumer catalog.
- **Immutable Contracts:** Every _products schema table comes with guaranteed schema contracts and refresh SLAs to downstream consumers.

7. FEDERATED MASTER DATA MANAGEMENT (MDM)

In a Data Mesh, every core master data entity (Customer, Material, Vendor) must be assigned a single Source of Truth domain owner. Other domains act as consumers and are barred from redefining that master data.

7.1 Master Data Ownership Map

Master Entity	Source of Truth Domain	Owner Table	SAP Source Tables
Customer Master	sap_domain_sales	sd_customers	KNA1, KNVV
Material Master	sap_domain_logistics	mm_materials	MARA, MAKT
Vendor Master	sap_domain_logistics	mm_vendors	LFA1, LFM1
G/L Accounts	sap_domain_finance	fi_accounts	SKA1, SKAT
Cost Centers	sap_domain_finance	co_cost_centers	CSKS, CSKT

Figure 7.1 — Federated MDM Ownership Map

7.2 Enforcement via Unity Catalog

- **Cross-Catalog FK Constraints:** Use Unity Catalog informational primary and foreign key constraints to build a formal map of how transactional tables reference master data.
- **IAM Governance:** Only the domain owner group (e.g., sales_data_engineers) has MODIFY permissions on master tables. Consumer domains get SELECT only.
- **Data Quality Gates:** Transactions with unmapped master data IDs are routed to a quarantine_rejections table. Domain owners commit to SLAs to resolve within 4 hours.

8. GOLDEN RECORD & MULTI-REGION HARMONIZATION

When ingesting from multiple SAP instances (e.g., S/4HANA in North America, ECC in Europe, legacy ERP in APAC), a Multi-Instance Consolidation Layer isolates raw ingestion by region and harmonizes into a single global data product.

8.1 Multi-Region Ingestion Architecture

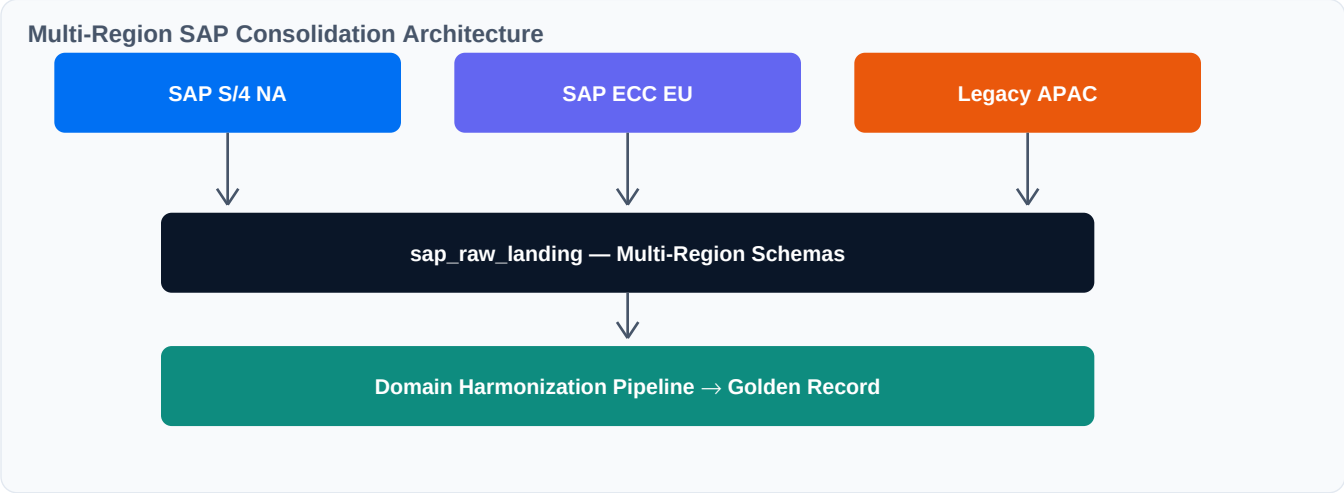

Figure 8.1 — Multi-Region Harmonization Flow

8.2 Golden Identifier Mapping

Local SAP System	Local ID	Enterprise Golden ID	Cleaned Name	Region
SAP_S4_NA	10002	CUST-992811-NA	Acme Corp US	NA
SAP_ECC_EU	84401	CUST-992811-EU	Acme Logistics BV	EU
Legacy_APAC	A-50009	CUST-992811-AP	Acme Asia Ltd	APAC

Figure 8.2 — Golden Identifier Cross-Reference Example

8.3 Harmonization Pipeline Steps

- **System Tagging:** Append source_system column to every record (e.g., SAP_S4_NA, SAP_ECC_EU, Legacy_APAC)
- **Schema Alignment:** Map varying column names across instances (e.g., S/4 ACDOCA vs ECC BSEG)
- **Golden ID Generation:** Apply Enterprise MDM-approved deterministic hashing (SHA-256) to resolve duplicate keys into a single global_customer_id

9. DELTA SHARING FOR EXTERNAL PARTNERS

Delta Sharing is an open protocol integrated into Unity Catalog that allows secure data sharing with external partners, sub-vendors, or logistics providers without copying data — regardless of whether the recipient uses Databricks.

9.1 Delta Sharing Architecture

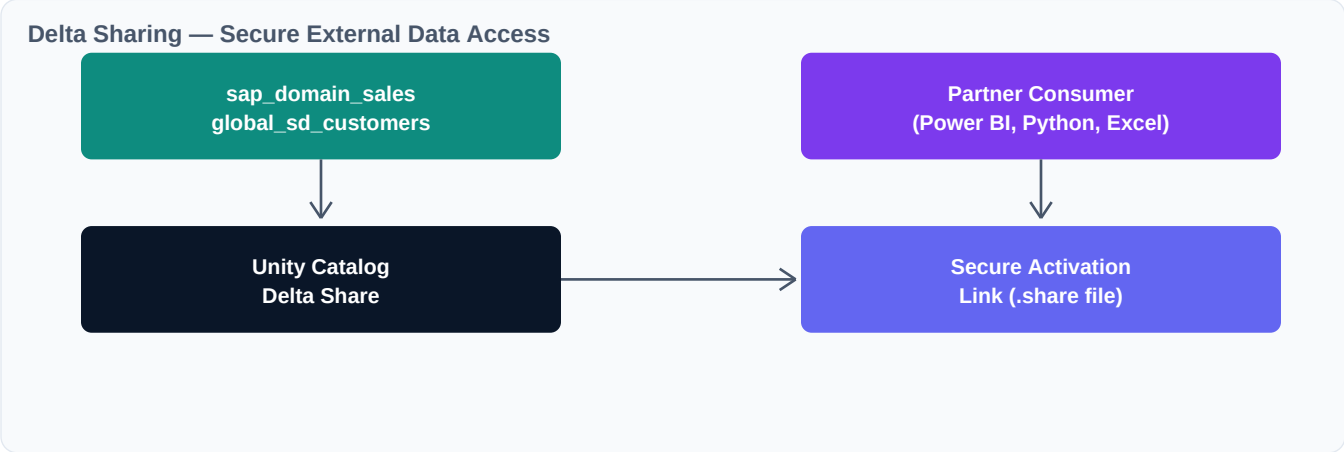

Figure 9.1 — Delta Sharing Architecture

- **Zero-Copy Sharing:** Data never leaves the Ball Corp S3/ADLS storage — recipients query via authenticated HTTPS/REST endpoints
- **Activation:** Share recipients receive a .share credential file to configure their consumer tools
- **Governance:** Governed via Unity Catalog — all access is audited and revocable in real time
- **Cross-Platform:** Supports Power BI, Python, Spark, Snowflake, Excel — any tool supporting the Delta Sharing protocol

10. AZURE AI FOUNDRY + DATABRICKS CROSS-PLATFORM

The cross-platform architecture connects Databricks Unity Catalog with Azure AI Foundry to enable AI agents that can query enterprise data, generate insights, and take safe actions against SAP production systems.

10.1 Cross-Platform Architecture

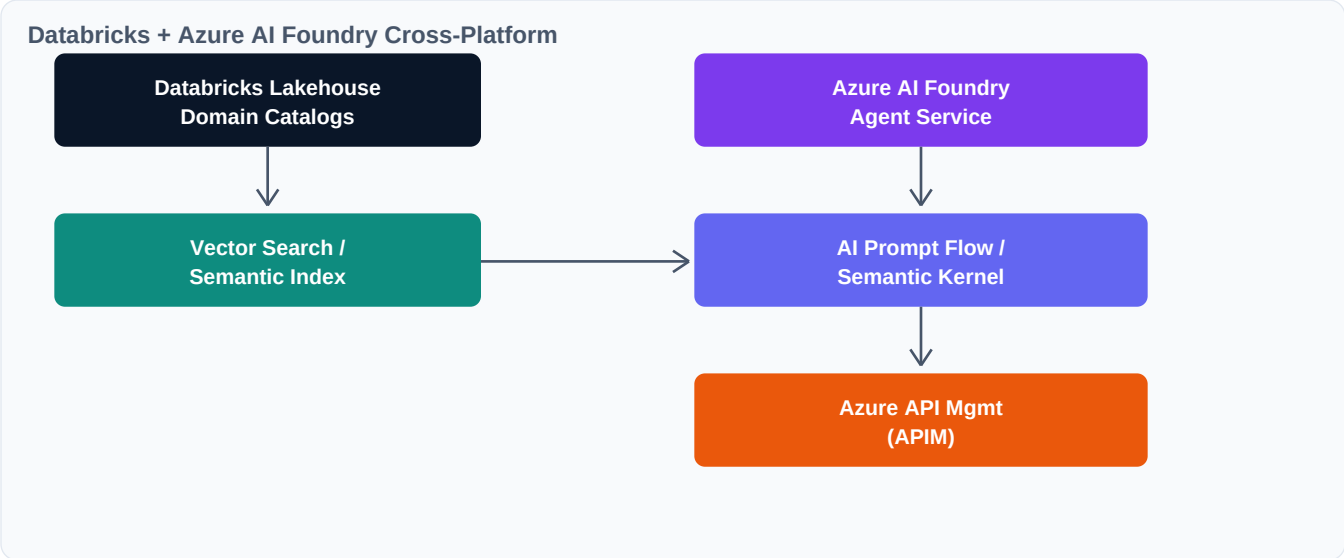

Figure 10.1 — AI Foundry Integration Architecture

10.2 Data Feeding Strategies

Pathway A — RAG (Retrieval Augmented Generation): Compute embeddings within Databricks Vector Search using SAP structural dictionary, text tables, and business glossaries. Azure AI Foundry queries the Vector Search REST API to inject SAP context into LLM prompts.

Pathway B — Text-to-SQL: For structural queries ('Show all open sales orders over $50K'), the AI agent generates SQL against a schema guide view and executes via Databricks Serverless SQL Warehouse using an Azure Service Principal.

10.3 Safe Action Execution

- **Tool Definitions:** OpenAPI specification stored in sap_enterprise.agent_governance.action_api_spec defines available system tools for AI agents.
- **Action Intent:** Agent generates structured JSON with extraction variables (customer_id, reason_code, requestor_id)
- **Guardrail Check:** Python validation wrapper queries agent_execution_limits table to verify compliance and security thresholds
- **Execute via APIM:** Validated requests route through Azure APIM which translates JSON into authenticated SAP OData/BAPI RFC calls

11. SECURITY, GOVERNANCE, LINEAGE & COMPLIANCE

11.1 Governance Framework — Four Pillars

Pillar	Scope	Implementation
Data Ownership	Domain-level RACI, steward assignments	Unity Catalog OWNER grants per catalog, quarterly reviews
Access & Security	RBAC, column masking, row-level security	SQL GRANT/REVOKE, masking functions, Private Link
Classification & Privacy	PII tagging, GDPR, retention policies	Column tags (confidential, PII), automated classification
Quality & Standards	DQ expectations, SLAs, naming conventions	DLT Expectations, quarantine tables, cross-domain SLAs

Figure 11.1 — Governance Framework Pillars

11.2 Data Lineage

Unity Catalog provides automatic table-level and column-level lineage tracking. Every transformation from Bronze → Silver → Gold is traced, showing exact data provenance from SAP source tables through to Gold analytics views.

11.3 Compliance Requirements

Regulation	Requirement	Implementation
SOX	7-year financial data retention	Delta Lake time-travel + archival policies
GDPR	Right to erasure, data residency	Column masking, region-specific catalogs, deletion pipelines
CCPA	Consumer data transparency	PII tagging, data lineage traceability via Unity Catalog

Figure 11.2 — Compliance Implementation Matrix

12. DATA QUALITY OBSERVABILITY & SLAs

Data quality is enforced at every layer of the Medallion architecture using Databricks DLT Expectations, automated quarantine queues, and cross-domain SLA commitments.

12.1 Quality Framework

- **DLT Expectations:** Apply Databricks Expectation frameworks to every public data product (e.g., EXPECT customer_id IS NOT NULL)
- **Quarantine Queues:** Transactions with unmapped or mismatched master data IDs are automatically routed to a domain-specific quarantine_rejections table
- **Cross-Domain SLAs:** Domain owners commit to resolving quarantined errors within set timeframes (e.g., 4 hours for customer master, 24 hours for material master)
- **Observability:** Automated freshness, completeness, and accuracy monitoring with PagerDuty/Teams alerting

12.2 Quality Metrics Dashboard

Metric	Bronze Target	Silver Target	Gold Target
Completeness	> 95%	> 99%	100%
Freshness	< 2 hours	< 1 hour	< 30 min
Accuracy	N/A (raw)	> 98%	> 99.5%
Uniqueness	N/A	> 99.9%	100% (deduped)
Schema Conformity	100%	100%	100%

Figure 12.1 — Data Quality SLA Targets by Medallion Layer

13. IMPLEMENTATION ROADMAP & RACI MATRIX

13.1 Phased Roadmap

Phase	Timeline	Key Deliverables
Phase 1: Foundation	Month 1-3	Deploy Unity Catalog + Metastore SAP FI/CO/SD batch extraction via ADF Bronze layer in ADLS Gen2 Basic governance policies + RBAC
Phase 2: Expand	Month 4-6	JDE + remaining SAP modules (MM, PP, QM) Silver layer transformations (DLT) Enable CDF on Delta tables Data quality framework rollout
Phase 3: Real-Time & ML	Month 7-9	FactoryTalk IoT streaming ingestion Gold layer dimensional models Feature Store + MLflow setup First ML models (defect detection)
Phase 4: Agentic AI	Month 10-12	Azure AI Foundry integration Vector Search + RAG pipeline Delta Sharing for partners Full governance operating model

Figure 13.1 — 12-Month Implementation Roadmap

13.2 RACI Matrix

Activity	CDO	Data Eng	Domain Steward	Platform Team	Security
ADF Pipeline Setup	I	R	C	A	C
Unity Catalog Deploy	I	R	C	A	R
Domain Schema Design	C	R	A	C	I
Data Quality Rules	I	R	A	C	I
Access Policy Definition	A	C	R	C	R
MDM Golden ID Setup	A	R	R	C	I
Delta Sharing Config	A	R	C	R	R
AI Foundry Integration	A	R	C	R	R
Governance Reviews	A	C	R	C	R

Figure 13.2 — RACI Matrix (R=Responsible, A=Accountable, C=Consulted, I=Informed)

13.3 Key Architectural Tasks — Immediate Priorities

- **Managed Identity Federation:** Configure an Azure Managed Identity spanning both Azure AI Foundry and Databricks Unity Catalog for unified RBAC without rotating API keys.
- **Meta-Data Views:** Assemble English translation tables for target SAP modules (merging text files with operational tables) to create clean, human-readable semantic layers.
- **APIM for SAP:** Configure Azure API Management gateway linked securely to SAP application router for the API endpoints that AI Foundry agents will trigger.
- **Governance Kickoff:** Establish domain data steward groups, assign catalog ownership, and implement the initial GRANT/REVOKE security templates.